

第二讲：计算——Julia 与 VFI

异质性主体宏观经济学的计算方法

孙振越 (Jeffrey Sun)

2026 年 5 月 12 日

多伦多大学

准备工作

贝尔曼方程

$$V(b, z) = \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + z, z) \quad \forall b, z$$

b 是财富。 z 是主体的生产率（暂时设为常数）。

该方程刻画了求解家庭问题所需的函数 V 。

✓ 唯一且可计算

对于任意有界的 v_1, v_2 ,

$$\|\mathcal{T}v_1 - \mathcal{T}v_2\|_\infty \leq \beta \|v_1 - v_2\|_\infty.$$

非正式地说，巴拿赫不动点定理意味着：

- 存在唯一的 V^* 满足 $V^* = \mathcal{T}V^*$ 。
- 值函数迭代总能找到 V^* 。

本讲：用有限网格替代连续状态 b ，将 $\mathcal{T}$ 写成一个 Julia 函数，迭代至不动点，并绘制 V_0 。

Julia

数组

```
xs = [1.0, 2.0, 3.0]      # Vector{Float64}, 长度为 3
M  = [1 2; 3 4]           # Matrix{Int64}, 2×2
zs = zeros(3, 4)          # 3×4 的全 0.0 矩阵
xs[1]                     # 1.0 --- 从 1 开始索引!
M[2, 1]                   # 3
```

- 索引从 1 开始。
- `xs[end]` 是最后一个元素。`xs[2:3]` 是一个切片。
- 数组按列优先存储：维度 1 是行，维度 2 是列。“向量”是列向量。

广播 (Broadcasting)

```
xs = [1.0, 2.0, 3.0]

xs .+ 10      # [11.0, 12.0, 13.0]
log.(xs)      # [0.0, 0.693, 1.099]
xs .^ 2        # [1.0, 4.0, 9.0]

# 对两个形状相同的数组逐元素运算：
[1,2,3] .* [10,20,30]  # [10, 40, 90]
```

点号 `.` 是“逐元素”的标记。

点号将任意函数或运算逐元素地施加，无需编写循环。

函数

```
function utility(c)
    c <= 0 && return -Inf
    return log(c)
end
```

简单条件判断的简写形式:

```
function utility(c)
    return c <= 0 ? -Inf : log(c)
end
```

单行函数同样有效:

```
u(c) = c <= 0 ? -Inf : log(c)
```

离散化

离散财富网格

- V 是定义在 $[0, \infty) \times W$ 上的函数。我们无法精确地表示它。
- 选取 N 个财富取值 $b_1 < b_2 < \dots < b_N$ 。
- 将 V 表示为向量 $[V(b_1), \dots, V(b_N)]$ 。
- 这使得该函数变为有限维。

注：值函数迭代的所有概念都**不依赖于**将 V 表示为向量。 V 可以是样条、神经网络，或定义在动态网格上，这些概念仍然成立。

线性间距与对数间距

```
loggrid(lo, hi, N) = exp.(range(log(lo), log(hi); length=N))

# 线性：等间距
b_grid = collect(range(0.1, 20.0; length=200))

# 对数：在借贷约束附近更稠密
b_grid = loggrid(0.1, 20.0, 200)
```

策略函数在 b 较小时非线性更强，因此我们采用对数间距，在较小的 b 上布置更多网格点。

效用

```
u(c) = c <= 0 ? -Inf : log(c)
```

更一般的形式是带 γ 的 CRRA 效用，但现在先保持简单。

将贝尔曼算子 $\mathcal{T}$ 写成 Julia 函数

```
function bellman_operator(V, params)
    # 从 `params` 中解包参数
    (;  $\beta$ , R, z, b_grid) = params

    # 计算达到每个下期财富水平 `b_next` 所需的储蓄水平 `b_end`
    b_end = (b_grid' .- z)./R

    # 在 `b_next` 维度（维度 2，即列）上对条件效用 `u.(b_grid .- b_end) .+  $\beta$ .*V` 求最大值。
    return maximum(u.(b_grid .- b_end) .+  $\beta$ .*V'; dims=2) |> vec
end
```

`b_grid` 是列向量。`b_grid'` 是行向量。

每一行对应一个 b 值；每一列对应一个 b' 值。最后一行在每个 b **内部**（按行）、**跨越** b' （按列）取 $\max$ 。

关于示例的说明

笔记本中的代码**并未**经过优化，也不追求速度。它们只是用来阐释概念。
我在 HouseholdStages.jl 包中提供了快得多的实现。

值函数迭代

值函数迭代

1. 以 $V_0 \equiv 0$ 起步（任何初始猜测都可以）。
2. $V_{k+1} \leftarrow \mathcal{T}V_k$ 。
3. 当 $\|V_{k+1} - V_k\|_\infty < \text{tol}$ 时停止。

solve_vfi

```
function solve_vfi(params; rtol=1e-6, verbosity=0)
    # 给出 V 的初始猜测
    V = zeros(size(params.b_grid))

    # 反复施加贝尔曼算子直至收敛
    Δ = Inf
    while Δ >= rtol
        # 更新 V
        TV = bellman_operator(V, params)

        # 计算误差
        Δ = maximum(abs.(TV .- V))

        # 更新 V
        V = TV
    end

    return V
end
```

基准校准

```
b_grid = loggrid(0.05, 40.0, 400)
params = (;β=0.96, R=1.04, z=1.0, b_grid)

V = solve_vfi(params)
```

- $\beta R < 1$, 否则 V^* 为无穷大。
- 网格：从 0.05 到 40 的 400 个对数间距点。
- `params` 是一个 `NamedTuple`（具名元组）。末尾的 `b_grid` 等价于 `b_grid=b_grid`。

值函数的特征

V 的形状与 u 的形状相似¹,

- $V(b)$ 是递增的 (财富越多 $\Rightarrow$ 未来消费越多)。
- $V(b)$ 是凹的: 增加一单位财富, 对穷人的价值高于对富人的价值。
- 其斜率是**财富的边际价值**: 由包络定理, $V'(b) = u'(c^*(b))$ 。

¹Sheeran (2017)

策略函数 c^*

$$c^*(b) = \arg \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + z).$$

```
function policy_function(V, params)
    (; β, R, z, b_grid) = params

    # 与 bellman_operator 的设置相同
    b_end = (b_grid' .- z) ./ R

    # 在 b' (列) 上取 argmax —— 为每一行取出对应的列索引
    idx = argmax(u.(b_grid .- b_end) .+ β .* V'; dims=2)
    return vec(getindex.(idx, 2))::Vector{Int}
end
```

与 `bellman_operator` 相同，只是用 `argmax` 取代了 `maximum`。
返回一个整数向量：对每个（行） b 给出最优 b' 的网格索引。